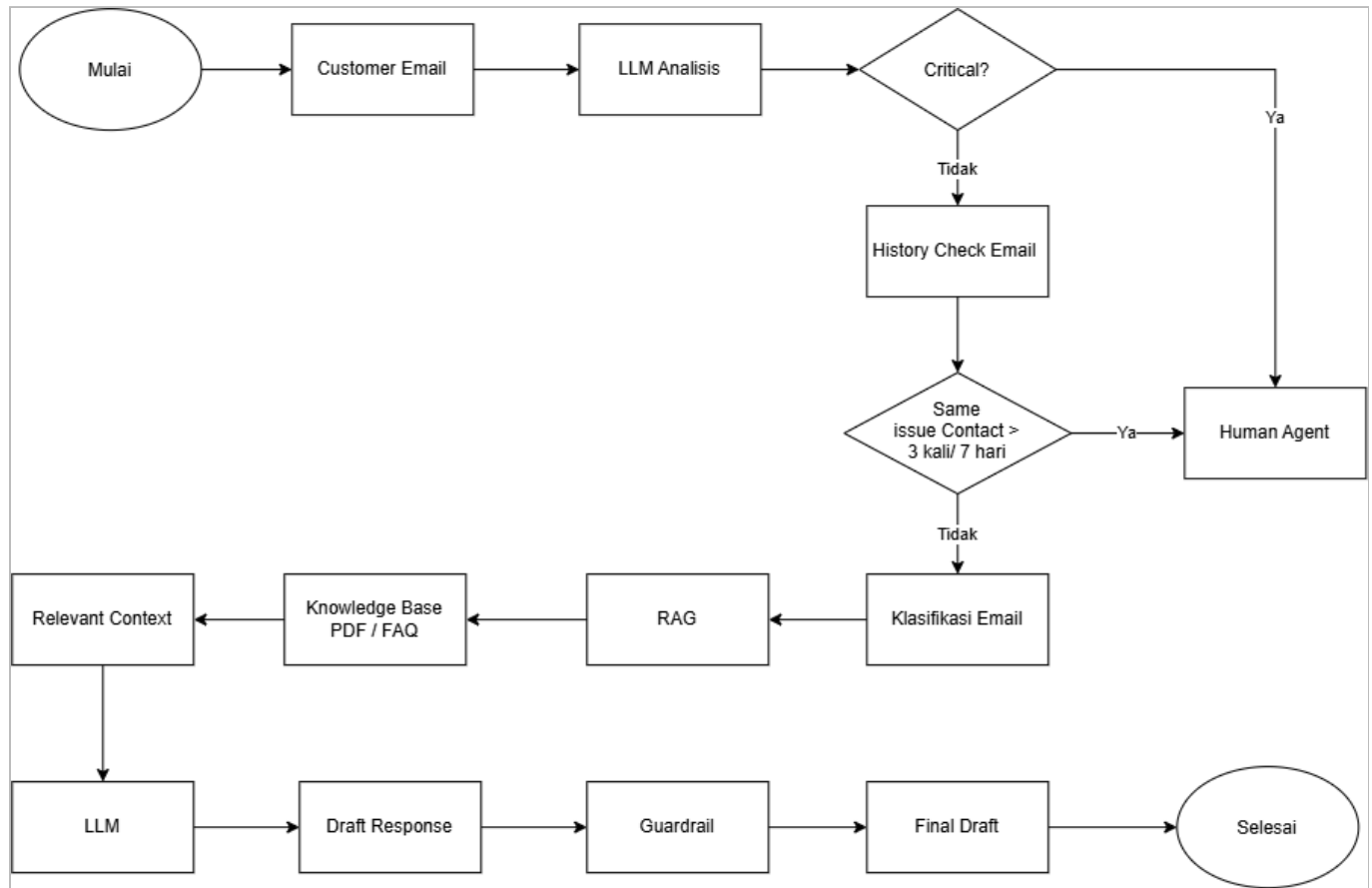


♦ Penjelasan Task 1

Alur penanganan email customer dengan LLM, RAG, dan guardrail.



Alur Task 1

01 — Analisis email & isu kritis

Pertama alurnya dimulai dari **email customer yang masuk ke sistem**. Setelah itu email akan masuk ke **LLM Analisis**. Di bagian ini LLM membaca isi email untuk mengetahui sebenarnya customer sedang membahas masalah apa.

Setelah itu saya taruh pengecekan **critical issue** terlebih dahulu. Contohnya seperti **data loss, service outage, atau security breach**. Menurut saya bagian ini perlu dicek lebih awal, karena kalau masalahnya memang cukup serius, tidak perlu menunggu customer menghubungi berkali-kali dulu. Walaupun baru satu kali mengirim email, kasus seperti ini lebih baik langsung diarahkan ke **human agent**.

02 — Pengecekan riwayat email

Kalau ternyata bukan masalah critical, baru sistem melakukan **history check email**. Di sini sistem melihat apakah customer sebelumnya pernah menghubungi untuk masalah yang sama.

Awalnya saya terpikir cukup pakai aturan **tiga kali menghubungi dalam satu minggu**, tapi setelah dipikirkan lagi ternyata kurang tepat. Bisa saja customer kirim tiga email dalam seminggu tetapi topiknya berbeda-beda. Kalau langsung dilempar ke human agent hanya karena jumlah emailnya, menurut saya jadi kurang efektif.

Karena itu saya ubah logikanya menjadi **tiga kali menghubungi dalam satu minggu dengan masalah yang sama**. Jadi sistem tidak cuma menghitung berapa kali customer kirim email, tetapi melihat apakah memang masalahnya

masih sama dan belum selesai.

Kalau ternyata sudah tiga kali membahas masalah yang sama dalam satu minggu, baru saya arahkan ke **human agent**. Alasannya karena kemungkinan jawaban sebelumnya belum menyelesaikan masalah customer, jadi lebih baik ditangani langsung oleh manusia.

03 — Klasifikasi & RAG

Kalau tidak, proses lanjut ke **klasifikasi email**. Di sini email dikelompokkan berdasarkan jenis masalahnya, misalnya **billing, technical issue, feedback**, atau kategori lainnya.

Setelah itu masuk ke **RAG**. Secara sederhana, RAG saya gunakan untuk mencari informasi yang sesuai dari **knowledge base**, misalnya dari FAQ, PDF, dokumentasi, atau informasi resmi perusahaan. Jadi LLM tidak langsung membuat jawaban sendiri, tetapi punya sumber informasi yang bisa dijadikan dasar.

Dari proses RAG itu, sistem akan mengambil **relevant context**, yaitu informasi yang paling sesuai dengan masalah customer. Context tersebut kemudian diberikan ke LLM berikutnya untuk membuat **draft response**.

04 — Respons & validasi

Jadi dua LLM di alur ini fungsinya berbeda. LLM yang pertama untuk **membaca dan memahami masalah email**, sedangkan LLM yang kedua untuk **membuat jawaban berdasarkan context yang didapat dari RAG**.

Setelah draft response dibuat, hasilnya masuk ke **guardrail**. Di bagian ini respons dicek lagi sebelum menjadi jawaban akhir, misalnya untuk memastikan isi jawabannya masih sesuai, tidak keluar dari informasi yang tersedia, dan tidak memberikan informasi yang seharusnya tidak diberikan.

Kalau sudah lolos, baru menjadi **final response** dan proses selesai.

✦ Penjelasan Task 2

Technical Implementation — memperbaiki scraping halaman kompleks, menangani konten panjang, dan memastikan ringkasan tetap singkat.

01 — Script awal

Script awal dibuat sederhana dan masih masuk akal untuk halaman website biasa.

```
async function scrapeAndSummarize(url: string) {
  const response = await fetch(url);
  const html = await response.text();

  // Mengambil seluruh teks dari HTML
  const text = html
    .replace(/<[^>]*>/g, " ")
    .replace(/\s+/g, " ")
    .trim();

  const prompt = `
  Ringkas isi website berikut:
```

```
    ${text}
    `;

    const summary = await callLLM(prompt);

    return summary;
}
```

Awalnya saya membuat script yang cukup sederhana. Website diambil menggunakan fetch, lalu tag HTML dihapus dan semua teks yang tersisa langsung dikirim ke LLM untuk dibuat ringkasannya.

Untuk halaman yang sederhana, cara ini mungkin masih bisa berjalan. Tetapi ketika halaman website lebih kompleks atau memiliki konten yang panjang, mulai muncul beberapa masalah.

02 — Bottleneck yang ditemukan

Masalah pertama: proses scraping

Walaupun tag HTML sudah dihapus, bukan berarti teks yang didapat sudah bersih. Masih bisa ada isi dari:

- navigation
- footer
- sidebar
- menu
- iklan
- script
- bagian lain yang sebenarnya tidak penting untuk diringkas

Jadi LLM bisa menerima banyak teks yang tidak berhubungan dengan isi utama halaman.

Masalah kedua: panjang konten

Pada script awal, seluruh teks langsung dikirim ke LLM dalam satu kali proses. Kalau website memiliki konten yang sangat panjang, input bisa menjadi terlalu besar dan hasil summary juga bisa menjadi kurang fokus atau terlalu panjang.

Bottleneck utama: scraping terlalu banyak mengambil content yang tidak diperlukan, dan long content langsung dikirim ke LLM sekaligus.

03 — Perbaiki scraping

Untuk perbaikannya, saya tidak lagi menghapus HTML hanya menggunakan regex. Saya menggunakan Cheerio supaya bisa memilih bagian halaman yang memang diperlukan.

```
import * as cheerio from "cheerio";

async function scrapePage(url: string) {
    const response = await fetch(url);

    if (!response.ok) {
        throw new Error("Gagal mengambil website");
    }

    const html = await response.text();
    const $ = cheerio.load(html);
```

```
// Menghapus bagian yang tidak diperlukan
$("script, style, nav, footer, header, aside").remove();

// Mengambil main content terlebih dahulu
const content = $("main").text() || $("article").text() || $("body").text();

return content.replace(/\s+/g, " ").trim();
}
```

Di bagian scraping saya ubah supaya sistem tidak mengambil seluruh teks secara mentah. Saya menggunakan Cheerio supaya bisa membuang elemen seperti navigation, footer, script, dan sidebar. Setelah itu sistem lebih memprioritaskan isi dari main atau article. Kalau tidak ditemukan, baru mengambil dari body.

04 — Menangani konten panjang

Setelah konten sudah lebih bersih, saya tambahkan proses untuk membagi teks menjadi beberapa bagian.

```
const MAX_CHUNK_WORDS = 1000;

function splitIntoChunks(text: string) {
  const words = text.replace(/\s+/g, " ").trim().split(" ");
  const chunks: string[] = [];

  for (let i = 0; i < words.length; i += MAX_CHUNK_WORDS) {
    chunks.push(words.slice(i, i + MAX_CHUNK_WORDS).join(" "));
  }

  return chunks;
}
```

Kemudian setiap bagian diringkas terlebih dahulu.

```
async function summarizeLongContent(text: string) {
  const chunks = splitIntoChunks(text);
  const summaries: string[] = [];

  for (const chunk of chunks) {
    const summary = await summarizeText(chunk);
    summaries.push(summary);
  }

  const combinedSummary = summaries.join("\n");
  return summarizeText(combinedSummary);
}
```

Untuk konten yang panjang, saya tidak langsung mengirim seluruh isi website ke LLM. Saya bagi dulu menjadi beberapa chunk berdasarkan jumlah kata supaya pemotongannya tidak terjadi di tengah kata. Masing-masing chunk diringkas, lalu hasil ringkasannya digabung dan diringkas sekali lagi menjadi satu final summary.

05 — Guardrail ringkasan

Karena soal juga meminta agar summary tetap concise, saya tambahkan guardrail. Pertama dari prompt:

```
async function summarizeText(text: string) {
  const prompt = `
Ringkas konten berikut.
Fokus hanya pada informasi utama.
Maksimal 150 kata.
```

```
Content:
${text}
`;

return callLLM(prompt);
}
```

Lalu saya tetap tambahkan pengecekan dari sisi program.

```
const MAX_SUMMARY_WORDS = 150;

function applyGuardrail(summary: string) {
  const words = summary.trim().split(/\s+/);

  if (words.length <= MAX_SUMMARY_WORDS) {
    return summary;
  }

  return words.slice(0, MAX_SUMMARY_WORDS).join(" ") + "...";
}
```

Saya tidak hanya meminta LLM membuat summary maksimal 150 kata, tetapi saya juga tetap mengecek hasil akhirnya dari program. Jadi kalau LLM masih menghasilkan jawaban lebih panjang dari batas tersebut, guardrail akan membatasi outputnya.

06 — Perubahan alur

Sebelum

```
Fetch Website
  ↓
Ambil Semua Teks
  ↓
LLM
  ↓
Summary
```

Sesudah

```
Fetch Website
  ↓
Parse HTML
  ↓
Buang Elemen yang Tidak Dibutuhkan
  ↓
Ambil Main Content
  ↓
Konten Panjang?
  ↓
Bagi Menjadi Chunk
  ↓
Ringkas Setiap Chunk
  ↓
Gabungkan Hasil
  ↓
Final Summary
  ↓
Guardrail
```

07 — Hasil pengujian

Setelah kedua script selesai, saya mencoba menjalankannya menggunakan website `https://info.cern.ch/`. Website tersebut saya pilih sebagai contoh halaman sederhana supaya hasil dari script awal dan script setelah perbaikan bisa dibandingkan secara langsung.

```
THINBOOK@DESKTOP-2USOHQ9 MINGW64 /d/New folder/New folder/Task2
$ npm run webscrape -- https://info.cern.ch/

> task2-web-summarizer@1.0.0 webscrape
> tsx Webscrape.ts https://info.cern.ch/

website `http://info.cern.ch` adalah halaman web pertama di dunia yang dibuat di CERN, laboratorium fisika tempat World Wide Web lahir.

Situs ini berfungsi sebagai pusat informasi historis bagi pengunjung untuk:
1. Menjelajahi tampilan website pertama di dunia.
2. Menggunakan simulator peramban berbasis teks (*line-mode browser*).
3. Mempelajari sejarah dan proses kelahiran World Wide Web.
4. Mengetahui lebih dekat tentang CERN, institusi tempat teknologi web pertama kali dikembangkan.

Situs ini dilestarikan sebagai monumen digital untuk mengenang asal-usul internet modern.

THINBOOK@DESKTOP-2USOHQ9 MINGW64 /d/New folder/New folder/Task2
$ npm run webscrape2 -- https://info.cern.ch/

> task2-web-summarizer@1.0.0 webscrape2
> tsx Webscrape2.ts https://info.cern.ch/

Situs http://info.cern.ch adalah halaman web pertama di dunia yang dibuat di laboratorium fisika CERN. Melalui situs ini, pengunjung dapat menjelajahi situs web pertama, menggunakan simulator *browser* mode-baris, mempelajari sejarah kelahiran web, dan mengenal laboratorium CERN.
```

Perbandingan hasil Webscrape dan Webscrape2

Pada bagian atas gambar, `Webscrape.ts` berhasil mengambil isi website dan menghasilkan ringkasan. Tetapi hasilnya masih cukup panjang dan beberapa informasi ditampilkan kembali dalam bentuk daftar.

Pada bagian bawah gambar, `Webscrape2.ts` menghasilkan jawaban yang lebih singkat dan langsung membahas informasi utama. Hal ini terjadi karena konten sudah dibersihkan terlebih dahulu, diproses melalui alur ringkasan yang lebih terkontrol, lalu dibatasi oleh guardrail.

Dari percobaan tersebut, kedua script sama-sama dapat digunakan. Perbedaannya adalah versi awal lebih cocok untuk halaman sederhana, sedangkan versi kedua lebih siap ketika struktur halaman lebih kompleks atau isi website lebih panjang.

✦ Penjelasan Task 3

Agent sederhana yang bisa menjawab berdasarkan satu dokumen, mengingat percakapan, dan menggunakan calculator jika diperlukan.

01 — Ide awal

Untuk task ini saya membuat sebuah agent sederhana yang bisa menjawab pertanyaan berdasarkan satu sample document. Saya menggunakan dokumen tentang **lowongan Frontend Developer di PT Nusa Digital** yang berisi informasi seperti tanggung jawab pekerjaan, teknologi yang digunakan, persyaratan, gaji, sistem kerja, dan proses seleksi.

Saya memilih menggunakan **raw Gemini API calls** karena alurnya masih cukup sederhana. Menurut saya belum perlu menggunakan framework seperti LangGraph atau CrewAI, karena di task ini hanya ada satu agent, satu dokumen, dan satu tool. Dengan cara ini alur program juga lebih mudah dibaca dan dijelaskan.

Sample document yang saya gunakan ada di file `'sample-document.md'`.

02 — Menjawab berdasarkan dokumen

Isi sample document saya baca dari file, lalu saya masukkan ke dalam **system instruction**. Di bagian tersebut saya memberi instruksi supaya agent hanya menggunakan dokumen sebagai sumber ketika menjawab pertanyaan tentang lowongan Frontend Developer.

Kalau informasi yang ditanyakan memang ada di dokumen, agent bisa langsung memberikan jawaban. Contohnya ketika user bertanya tentang teknologi yang digunakan, agent akan menjawab **HTML, CSS, TypeScript, React, dan Git**.

Kalau jawabannya tidak ada di dokumen, agent diarahkan untuk mengatakan bahwa informasi tersebut tidak tersedia. Saya menambahkan aturan ini supaya agent tidak membuat informasi sendiri di luar dokumen yang diberikan.

03 — Conversation memory

Requirement berikutnya adalah agent harus mempunyai **conversation memory**. Untuk bagian ini saya menyimpan pesan user dan jawaban model ke dalam array `history`.

Setiap ada pertanyaan baru, history tersebut dikirim lagi ke Gemini bersama pertanyaan terbaru. Dengan begitu agent masih bisa melihat konteks percakapan sebelumnya.

Saya juga menambahkan variabel `rememberedName` untuk menyimpan nama yang diperkenalkan oleh user. Contohnya seperti ini:

```
User : Nama saya Andi.  
Agent : Halo Andi, ada yang bisa saya bantu?  
  
User : Siapa nama saya?  
Agent : Nama Anda Andi.
```

Pada contoh tersebut, pertanyaan kedua tidak menyebutkan nama lagi. Tetapi agent tetap bisa menjawab karena nama dan percakapan sebelumnya masih tersimpan.

Memory yang saya buat masih bersifat **in-memory**, jadi hanya bertahan selama program dijalankan. Kalau program ditutup, history akan kembali kosong. Menurut saya ini sudah cukup untuk requirement agent sederhana. Kalau nantinya digunakan untuk aplikasi yang lebih besar, memory bisa disimpan ke database.

04 — Calculator tool

Selain memory, agent juga harus mempunyai sebuah tool yang hanya dipanggil ketika diperlukan. Saya memilih membuat **calculator tool** karena mudah menunjukkan perbedaan antara pertanyaan biasa dan pertanyaan yang membutuhkan perhitungan.

Calculator saya daftarkan ke Gemini sebagai **function declaration**. Di deskripsinya saya jelaskan bahwa calculator hanya digunakan untuk pertanyaan matematika. Setelah itu Gemini yang memutuskan apakah tool perlu dipanggil atau tidak.

Misalnya user bertanya:

Apa saja teknologi yang digunakan?

Untuk pertanyaan tersebut Gemini tidak perlu memanggil calculator karena jawabannya bisa langsung diambil dari dokumen.

Tetapi ketika user bertanya:

Berapa total gaji selama masa percobaan?

Gemini akan melihat dari dokumen bahwa gajinya adalah Rp7.500.000 per bulan dan masa percobaannya berlangsung selama tiga bulan. Setelah itu Gemini meminta program memanggil calculator dengan ekspresi:

7500000 * 3

Program menjalankan perhitungan tersebut, lalu hasilnya dikirim kembali ke Gemini untuk dibuat menjadi jawaban akhir. Hasilnya adalah total gaji selama masa percobaan sebesar **Rp22.500.000**.

05 — Pengecekan keamanan calculator

Untuk menjalankan perhitungan saya tidak menggunakan `eval`, karena `eval` bisa menjalankan kode JavaScript yang tidak seharusnya dijalankan.

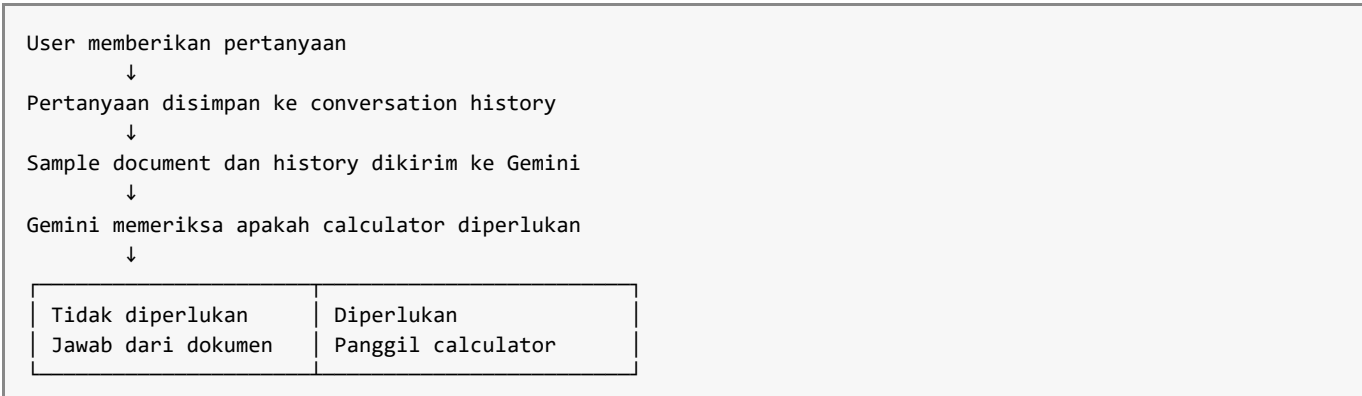
Sebagai gantinya, saya membuat parser aritmetika sederhana. Calculator hanya menerima:

- angka;
- tanda kurung;
- penjumlahan dan pengurangan;
- perkalian dan pembagian; serta
- operator modulo.

Kalau Gemini mengirim karakter atau ekspresi di luar aturan tersebut, calculator akan menolaknya. Jadi tool hanya bisa digunakan untuk perhitungan dan tidak bisa menjalankan kode lain.

06 — Alur program

Secara keseluruhan, alur agent yang saya buat seperti ini:



↓
Jawaban disimpan ke history
↓
Jawaban ditampilkan kepada user

Jadi pada implementasi ini Gemini mempunyai tiga tugas utama. Pertama, memahami pertanyaan user. Kedua, mencari jawaban berdasarkan sample document dan konteks percakapan. Ketiga, memutuskan kapan calculator memang perlu digunakan.

Sedangkan program tetap bertanggung jawab untuk menyimpan memory, menjalankan calculator, dan mengirim hasil tool kembali ke Gemini.
